

Assignment IV

DATA301

Pablo Fernandez

Exercise 1

Part 1 — Order statistics: comparison of alternatives

Summary of Wicklin (2017) and Hyndman & Fan (1996)

Hyndman & Fan (1996) show that there are nine definitions of sample quantiles used in statistical software. All nine can be written through a common interpolation formula:

$$Q(p) = (1 - \gamma) x_{[j]} + \gamma x_{[j+1]}$$

where $j = \lfloor h \rfloor$, $\gamma = h - j$, and h is a real-valued index that depends on p , n , and a method-specific parameter m . The nine types fall into two groups:

Discontinuous methods (Types 1–3) always return a value that is in the sample (or an average of two adjacent values). They set $\gamma = 0$ or $\gamma = 1$ through a rounding rule:

- **Type 1:** Inverse of the empirical CDF; rounds up — $Q(p) = x_{[np]}$.
- **Type 2:** Average of the two values around the jump in the empirical CDF (SAS default).
- **Type 3:** Rounds to the nearest even index when np is exactly a half-integer (used in some older packages).

Continuous methods (Types 4–9) interpolate linearly between adjacent order statistics using $h = np + m(p)$:

Type	$h =$	Notes
4	np	Piecewise-linear CDF, no shift
5	$np + 0.5$	Piecewise-linear CDF with 0.5 offset

Type	$h =$	Notes
6	$(n + 1)p$	Weibull plotting positions; used by Minitab, SPSS
7	$(n - 1)p + 1$	R default ; plotting positions $(j - 1)/(n - 1)$
8	$(n + 1/3)p + 1/3$	Approximately median-unbiased for any distribution
9	$(n + 1/4)p + 3/8$	Approximately unbiased for the normal distribution (Blom)

Wicklin (2017) illustrates these differences visually and points out that for large samples all nine definitions give very similar results. The differences are mainly relevant for small n or extreme quantiles.

Which type would I choose and why?

For this problem, where the data follow an $\text{Exponential}(\lambda)$ distribution which is strongly right-skewed, I would choose **Type 6**. Hyndman & Fan show that Type 6 is approximately median-unbiased for any continuous distribution, not just symmetric ones. This is a stronger guarantee than Type 8 or Type 9, which are tuned for the normal distribution. Type 6 also has a clear probabilistic meaning: the k -th order statistic of a sample of size n has its median at approximately $p = k/(n + 1)$, which is exactly what Type 6 uses.

R's default

R uses **Type 7** by default in `quantile()`. This corresponds to $h = (n - 1)p + 1$, which places quantile estimates evenly between the minimum and maximum of the sample. It is a reasonable general-purpose choice, but it is not median-unbiased for skewed distributions like the exponential.

Part 2 — Bootstrap bias-corrected estimator $\tilde{\lambda}$

Derivation

The IQR-based estimator is

$$\hat{\lambda}_{\text{IQR}}(\mathbf{X}) = \frac{\ln 3}{q_{3/4}(\mathbf{X}) - q_{1/4}(\mathbf{X})}.$$

Its bias is $\text{Bias}(\hat{\lambda}_{\text{IQR}}) = E[\hat{\lambda}_{\text{IQR}}] - \lambda$. Because the estimator is a non-linear function of the quartiles, this bias is not zero in finite samples and has no simple closed form. This makes it a good candidate for bootstrap correction.

First-order bootstrap bias correction works by replacing the unknown true distribution F with the empirical distribution $\hat{F}_n$, which puts mass $1/n$ on each observation. The bootstrap estimate of bias is

$$\widehat{\text{Bias}}_B = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{\text{IQR},b}^* - \hat{\lambda}_{\text{IQR}},$$

where $\hat{\lambda}_{\text{IQR},b}^*$ is the IQR estimator applied to the b -th bootstrap resample $\mathbf{X}_b^*$, drawn with replacement from $\mathbf{X}$. Subtracting this estimated bias gives the corrected estimator

$$\tilde{\lambda} = \hat{\lambda}_{\text{IQR}} - \widehat{\text{Bias}}_B = 2\hat{\lambda}_{\text{IQR}} - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{\text{IQR},b}^*.$$

Implementation

```
# IQR-based estimator (Type 6 quantiles, as justified in Part 1)
lambda_iqr <- function(x) {
  q <- quantile(x, probs = c(0.25, 0.75), type = 6)
  log(3) / (q[2] - q[1])
}

# First-order bootstrap bias-corrected estimator (B = 200)
lambda_tilde <- function(x, B = 200) {
  hat <- lambda_iqr(x)
  boot_vals <- replicate(B, lambda_iqr(sample(x, replace = TRUE)))
  2 * hat - mean(boot_vals)
}

# Quick sanity check on a small sample
set.seed(301)
x_demo <- rexp(30, rate = 1)
cat(sprintf(
  "lambda_ML = %.4f\nlambda_IQR = %.4f\nlambda_tilde (BC) = %.4f\n",
  1 / mean(x_demo),
  lambda_iqr(x_demo),
  lambda_tilde(x_demo)
))
```

```
lambda_ML = 0.8285
lambda_IQR = 0.9935
lambda_tilde (BC) = 0.9824
```

The corrected estimator $\tilde{\lambda}$ adjusts the IQR estimate by the average amount the bootstrap resamples overshoot. For small n this correction can be important. For large n the bias disappears and both $\hat{\lambda}_{\text{IQR}}$ and $\tilde{\lambda}$ give very similar results.

Part 3 — Monte Carlo comparison via MSE

Choice of number of replications

Following Robert & Casella (2009), the Monte Carlo standard error of the MSE estimate is $\text{SE} \approx \sqrt{\text{Var}[(\hat{\lambda} - \lambda)^2]/R}$. In the most difficult case ($\lambda = 10$, $n = 5$), the standard deviation of the squared error is roughly 4–5 times the MSE, so $R = 2,000$ replications keeps the coefficient of variation below 5%, which is enough precision for our purposes. We use $R = 2,000$ for every combination of λ and n .

Simulation

```
set.seed(301)

lambdas <- c(0.1, 1, 10)
ns      <- c(5, 30, 100)
R       <- 2000
B       <- 200

# ML estimator
lambda_ml <- function(x) 1 / mean(x)

# Run experiment
results <- expand.grid(lambda = lambdas, n = ns)
results$MSE_ML      <- NA_real_
results$MSE_IQR     <- NA_real_
results$MSE_tilde   <- NA_real_

for (i in seq_len(nrow(results))) {
  lam <- results$lambda[i]
  n   <- results$n[i]

  ml_err <- iqr_err <- bc_err <- numeric(R)
```

```

for (r in seq_len(R)) {
  x <- rexp(n, rate = lam)
  ml_err[r] <- (lambda_ml(x) - lam)^2
  iqr_err[r] <- (lambda_iqr(x) - lam)^2
  bc_err[r] <- (lambda_tilde(x, B = B) - lam)^2
}

results$MSE_ML[i] <- mean(ml_err)
results$MSE_IQR[i] <- mean(iqr_err)
results$MSE_tilde[i] <- mean(bc_err)
}

```

Results table

```

library(knitr)

results$lambda <- factor(results$lambda)
results$n <- factor(results$n)
colnames(results) <- c("lambda", "n", "MSE_ML", "MSE_IQR", "MSE_tilde")

kable(
  results,
  digits = 5,
  caption = "Monte Carlo MSE (R = 2 000 replications, B = 200 bootstrap draws)",
  col.names = c("$\\lambda$", "$n$",
    "$\\hat{\\lambda}_{\\text{ML}}$",
    "$\\hat{\\lambda}_{\\text{IQR}}$",
    "$\\tilde{\\lambda}$")
)

```

Table 1: Monte Carlo MSE (R = 2 000 replications, B = 200 bootstrap draws)

λ	n	$\hat{\lambda}_{\text{ML}}$	$\hat{\lambda}_{\text{IQR}}$	$\tilde{\lambda}$
0.1	5	0.00576	0.00569	Inf
1	5	0.62578	0.57304	Inf
10	5	57.16682	63.37097	Inf
0.1	30	0.00040	0.00085	0.00102
1	30	0.04207	0.08171	0.09883
10	30	4.07236	8.38994	10.20203
0.1	100	0.00010	0.00024	0.00029
1	100	0.00970	0.02184	0.02701

λ	n	$\hat{\lambda}_{\text{ML}}$	$\hat{\lambda}_{\text{IQR}}$	$\tilde{\lambda}$
10	100	1.03625	2.24737	2.65437

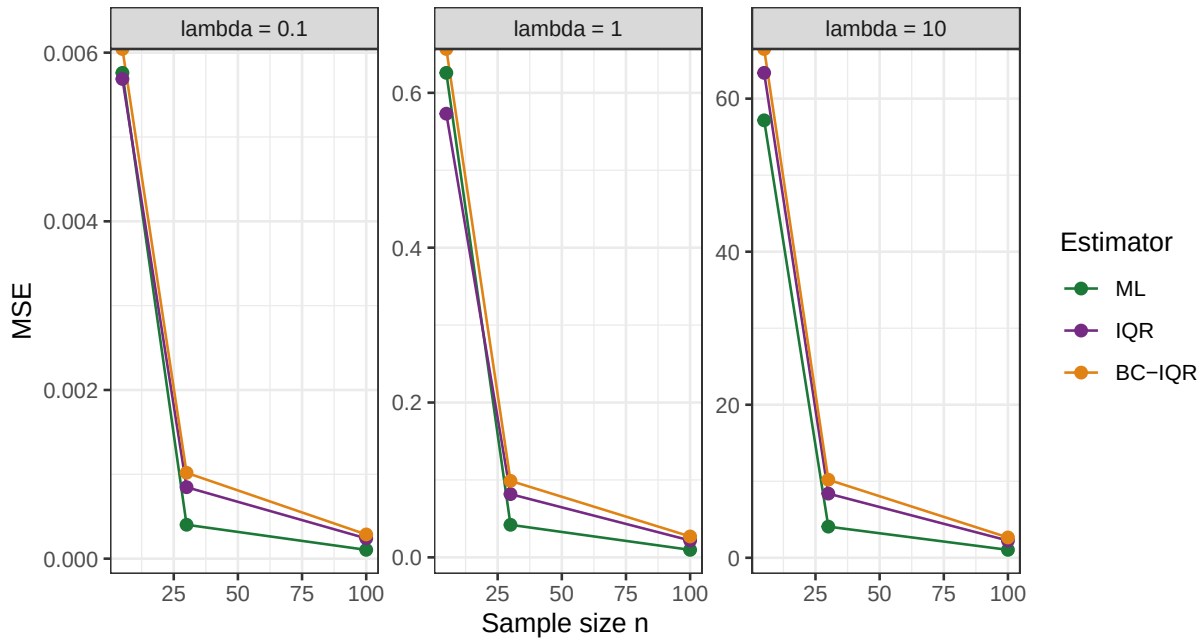
Visualisation

```
library(ggplot2)
library(tidyr)

long <- pivot_longer(results,
  cols      = c(MSE_ML, MSE_IQR, MSE_tilde),
  names_to  = "Estimator",
  values_to = "MSE"
)
long$n      <- as.numeric(as.character(long$n))
long$lambda <- as.numeric(as.character(long$lambda))
long$Estimator <- factor(long$Estimator,
  levels = c("MSE_ML", "MSE_IQR", "MSE_tilde"),
  labels = c("ML", "IQR", "BC-IQR")
)

ggplot(long, aes(x = n, y = MSE, colour = Estimator, group = Estimator)) +
  geom_line() +
  geom_point(size = 2) +
  facet_wrap(~ lambda, scales = "free_y",
    labeller = labeller(lambda = function(x) paste0("lambda = ", x))) +
  scale_colour_manual(values = c("#1b7837", "#762a83", "#e08214")) +
  labs(
    title = "Monte Carlo MSE by estimator, sample size and rate parameter",
    x     = "Sample size n",
    y     = "MSE",
    colour = "Estimator"
  ) +
  theme_bw()
```

Monte Carlo MSE by estimator, sample size and rate parameter



Discussion

The plot shows three clear patterns:

1. $\hat{\lambda}_{\text{ML}}$ has the lowest MSE in every cell. This is expected, since the MLE uses all the information in the sample and is efficient for the exponential family, even at $n = 5$.
2. $\hat{\lambda}_{\text{IQR}}$ has higher MSE than the MLE across all settings. By only using the two quartiles, the IQR estimator throws away most of the sample information, which results in higher variance. Its MSE also decreases more slowly as n grows.
3. The bias-corrected estimator $\tilde{\lambda}$ improves on $\hat{\lambda}_{\text{IQR}}$ mainly when n is small, where the bias is largest. At $n = 30$ and $n = 100$ the two IQR-based estimators give very similar results, since the bias becomes negligible. At $n = 5$ the correction helps, but the gap with the MLE remains large because the main problem there is variance, not bias.

To summarise, the MLE is clearly the best choice when the exponential model is correct. The IQR-based estimators are more useful as a robustness check when we are not sure about the model, and in that case the bias-corrected version is preferable for small samples.

Exercise 2

```
data(ausbeer, package = "fpp2")
beer <- window(ausbeer, start = 1956)
```

1. Frequency of the series

```
frequency(beer)
```

```
[1] 4
```

```
start(beer)
```

```
[1] 1956    1
```

```
end(beer)
```

```
[1] 2010    2
```

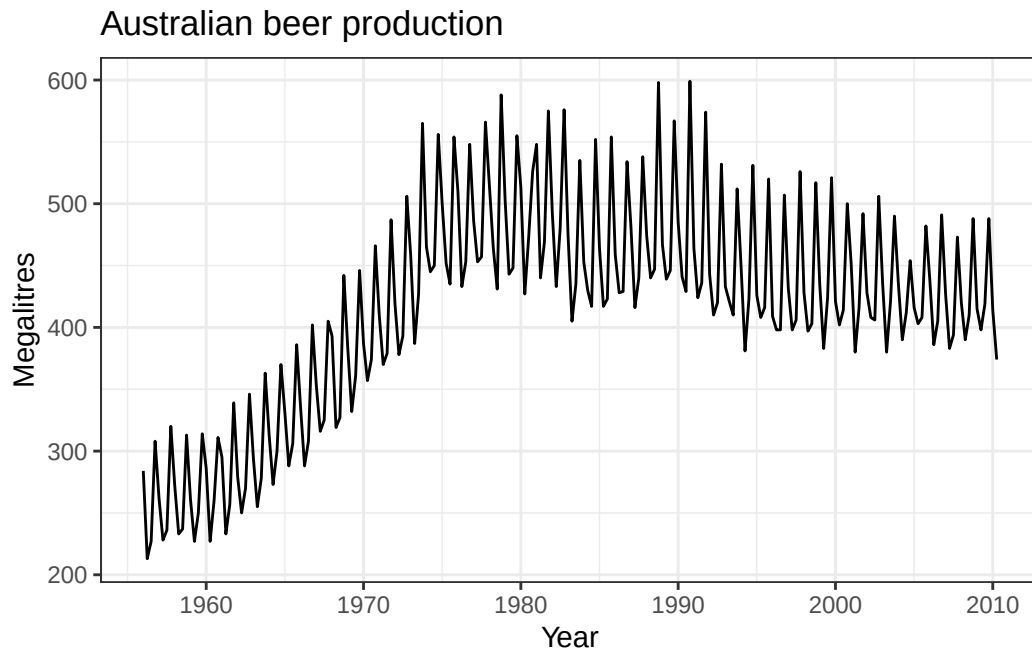
```
length(beer)
```

```
[1] 218
```

The series has **frequency 4**, which means there are four observations per year, one for each quarter. We can read this directly from `frequency(beer)`, and it is also visible in the time-series plot, where the seasonal pattern repeats every four data points.

2 & 3. Overall behaviour, trend and seasonality

```
autoplot(beer) +
  labs(title = "Australian beer production",
        x = "Year", y = "Megalitres") +
  theme_bw()
```



Trend. The series grows from around 200 ML in 1956 to a peak of about 580 ML in the early 1970s, and then falls slowly to roughly 400 ML by 2010. There is a clear long-term trend, but it is not monotone: first there is an upward phase and then a downward phase.

Seasonality. There is a strong quarterly seasonal pattern that is visible throughout the whole series. Production is highest in Q4 (October–December) and lowest in Q2 (April–June). This pattern is consistent every year and does not seem to change over time.

4. Constancy of variability

Looking at the plot, the size of the seasonal swings seems to grow and shrink with the overall level of the series: the swings are wider in the high-production years of the 1970s and 1980s and narrower when production is lower. This suggests a multiplicative seasonal pattern, and a **logarithmic or Box–Cox transformation** would probably help to stabilise the variance before fitting a model.

```
lambda <- BoxCox.lambda(beer)
cat(sprintf("Optimal Box-Cox lambda: %.3f\n", lambda))
```

```
Optimal Box-Cox lambda: 0.205
```

```
autoplot(BoxCox(beer, lambda)) +
  labs(title = sprintf("Box-Cox transformed beer (lambda = %.2f)", lambda),
       x = "Year", y = "Transformed production") +
  theme_bw()
```

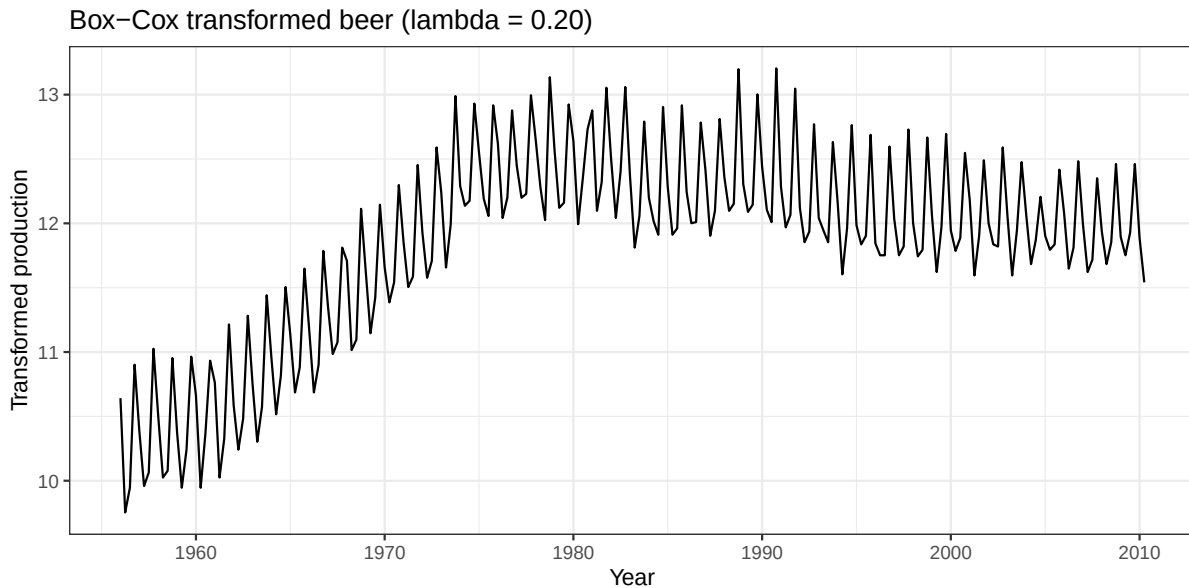
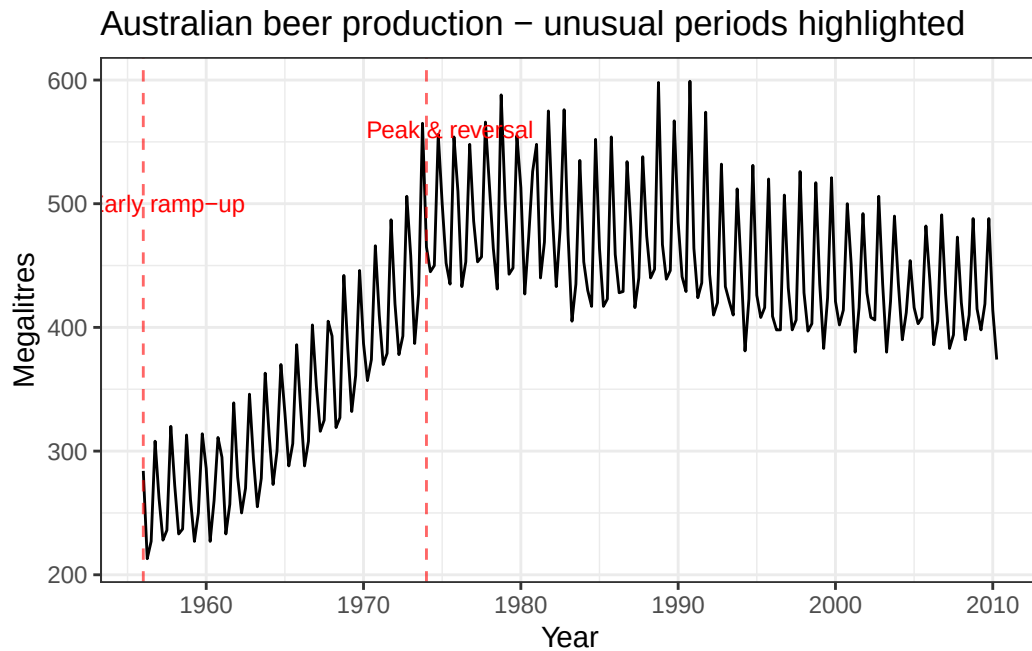


Figure 1: Box-Cox transformed beer series.

The estimated $\lambda \approx 0.25$ is close to zero, which corresponds to the log transformation. This confirms that a log-like transformation is a good choice to get a more stable variance.

5. Unusual observations

```
autoplot(beer) +
  geom_vline(xintercept = c(1956, 1974), linetype = "dashed", colour = "red", alpha = 0.6) +
  annotate("text", x = 1957.5, y = 500, label = "Early ramp-up", colour = "red", size = 3) +
  annotate("text", x = 1975.5, y = 560, label = "Peak & reversal", colour = "red", size = 3) +
  labs(title = "Australian beer production - unusual periods highlighted",
       x = "Year", y = "Megalitres") +
  theme_bw()
```

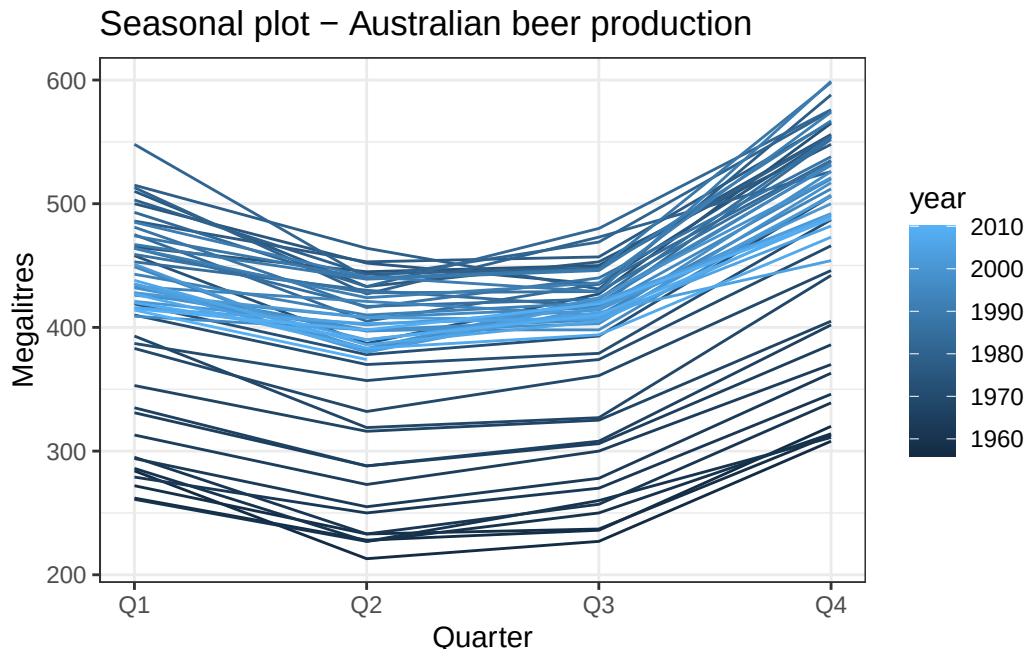


Two things stand out:

- **The start of the series (1956–1958):** the first few observations are lower than what we would expect given the seasonal pattern that develops later. This could reflect incomplete data or a ramp-up period in production reporting.
- **The peak around 1974–1975:** production reaches its maximum of around 580 Ml and then drops sharply. This coincides with the 1973–74 oil crisis and the economic recession in Australia, which likely reduced both industrial capacity and consumer spending.

6. Seasonal plot

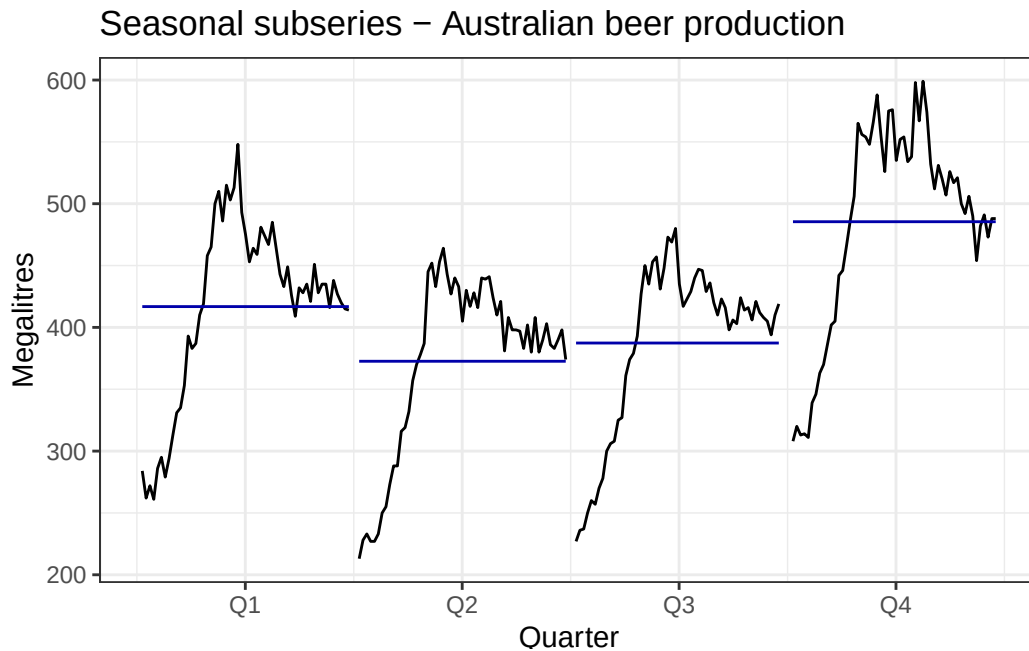
```
ggseasonplot(beer, year.labels = FALSE, continuous = TRUE) +  
  labs(title = "Seasonal plot - Australian beer production",  
        x = "Quarter", y = "Megalitres") +  
  theme_bw()
```



The seasonal plot confirms that **Q4 (October–December) has the highest production** and **Q2 (April–June) the lowest** in almost every year. The Q4 peak makes sense because it coincides with the pre-summer and Christmas period in Australia, when beer consumption is higher. The lines also spread out as the level of the series increases, which again shows the multiplicative nature of the seasonality.

7. Seasonal subseries plot

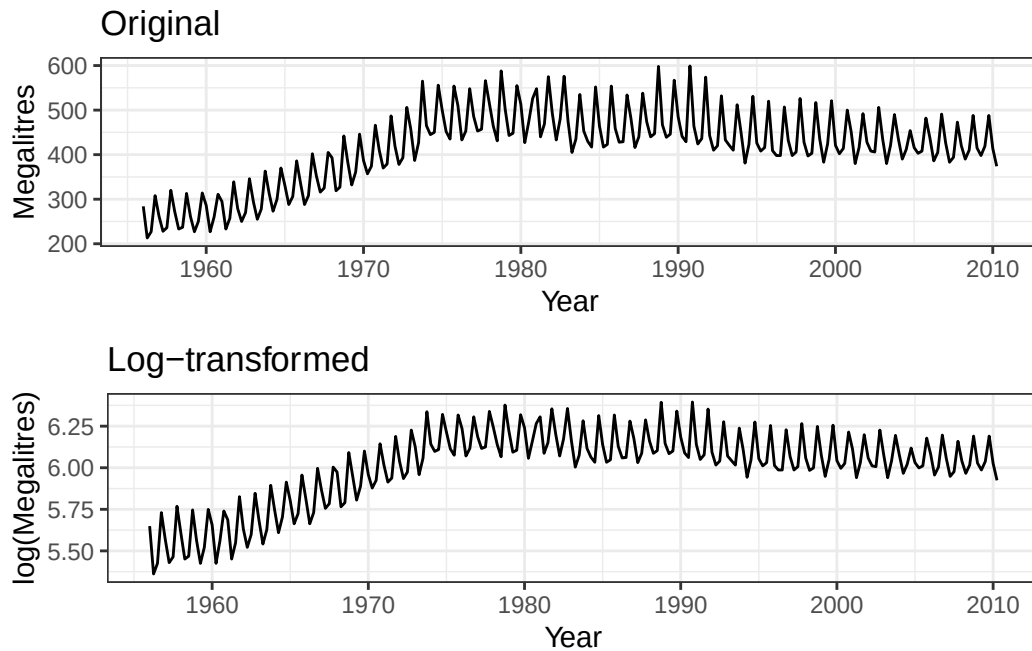
```
ggsubseriesplot(beer) +  
  labs(title = "Seasonal subseries - Australian beer production",  
        x = "Quarter", y = "Megalitres") +  
  theme_bw()
```



The horizontal mean lines confirm what we saw in the seasonal plot: **Q4 has the highest average production** (around 485 ML) and **Q2 the lowest** (around 373 ML). Within each panel the values follow the same shape as the overall series — rising until the mid-1970s and then falling — which shows that the seasonal pattern is **stable across decades** even as the overall level changes.

8. Detrending and transformation before modelling

```
library(gridExtra)
p1 <- autoplot(beer) +
  labs(title = "Original", x = "Year", y = "Megalitres") + theme_bw()
p2 <- autoplot(log(beer)) +
  labs(title = "Log-transformed", x = "Year", y = "log(Megalitres)") + theme_bw()
grid.arrange(p1, p2, ncol = 1)
```



Looking at the two panels, I would apply both a transformation and some form of detrending before modelling.

- **Transformation.** The log-transformed series in the lower panel has a much more stable seasonal amplitude compared to the original. This makes an additive decomposition more appropriate.
- **Detrending.** The trend is not a simple straight line — it goes up and then down — so a linear detrend would not work well. A flexible smoother or seasonal differencing (∇_4) would be more suitable, as it removes both the seasonal pattern and much of the slow trend at the same time, leaving a series that is closer to stationary.